Informe de Análisis del Sistema - Versión Ampliada

Carpeta 3: Factibilidad, Análisis Funcional, Sistema y Técnico

Autor: Yerson Stiven Cuellar Rubiano | **Programa:** Análisis y Desarrollo de Software (ADSO) | **Institución:** SENA Centro de la Industria (Neiva, Huila) | **Versión:** 2.0 (Detallada)

1. Introducción al Documento de Análisis

El presente documento constituye el núcleo de la fase de **Análisis del Sistema (Carpeta 3)**. Con el objetivo de cumplir rigurosamente con los estándares de ingeniería de software del SENA, este entregable unifica y profundiza en cuatro ejes fundamentales: el Estudio de Factibilidad multidimensional, el Análisis Funcional de procesos, la delimitación analítica del Sistema y el Análisis Técnico avanzado.

2. Estudio de Factibilidad Extendido (feasibility-study)

La viabilidad del proyecto de plataforma de domicilios ha sido evaluada exhaustivamente bajo cuatro dimensiones críticas para garantizar el éxito en su ejecución y posterior despliegue:

A. Factibilidad Técnica

El desarrollo se considera **Altamente Factible**. La elección de tecnologías basadas en JavaScript/TypeScript para todo el stack (Next.js para web, React Native para móvil y Node.js en funciones serverless) elimina la curva de aprendizaje de múltiples lenguajes. El uso de `Supabase` como Backend as a Service (BaaS) abstrae la complejidad de administrar servidores Linux, configurar firewalls o realizar respaldos manuales de PostgreSQL, permitiendo que un solo desarrollador cumpla con los tiempos del proyecto.

B. Factibilidad Económica

El proyecto es **Económicamente Viable** con una inversión inicial requerida de $0 COP durante la fase de desarrollo y primeros meses de despliegue. Esto se logra aprovechando las capas gratuitas (Free Tiers) empresariales:

- **Infraestructura Backend (Supabase):** Hasta 50,000 usuarios activos mensuales (MAU) y 500MB de base de datos sin costo.
- **Alojamiento Frontend (Vercel):** Despliegue automatizado con ancho de banda de 100GB mensuales gratuitos.
- **Herramientas API:** Wompi ofrece un entorno *Sandbox* gratuito para pruebas de pago, y Google Maps API otorga un crédito mensual recurrente de $200 USD, suficiente para miles de cargas de mapas.

C. Factibilidad Operativa

El sistema se adaptará sin fricción a la operación diaria de los restaurantes. El panel web implementa componentes UI accesibles (shadcn/ui), garantizando que el personal de caja o administración no requiera capacitaciones extensas. Para los domiciliarios, la app móvil está diseñada para operar con "un solo pulgar", permitiendo cambiar estados ("En camino", "Entregado") con botones grandes y retroalimentación háptica.

D. Factibilidad Legal (Añadido para rigor académico)

El sistema cumplirá con la normatividad colombiana de protección de datos personales (**Ley Estatutaria 1581 de 2012 - Habeas Data**). Se implementarán casillas de verificación para la aceptación de términos, y las bases de datos encriptarán la información sensible (contraseñas y datos financieros tokenizados) para proteger la privacidad del consumidor.

3. Análisis Funcional y Transformación de Datos (functional-analysis)

El análisis funcional modela exactamente cómo el sistema recibe información cruda, la procesa a través de la lógica de negocio y genera resultados de valor.

Módulo / Proceso	Datos de Entrada (Inputs)	Transformación y Lógica Interna	Salida Esperada (Outputs)
1. Gestión de Catálogo y Menú	Archivos de imagen (JPG/PNG), cadenas de texto (nombre, descripción), valores numéricos (precio, stock).	El backend comprime y sube la imagen a Supabase Storage, genera una URL pública, e inserta el registro relacional en la tabla `productos`.	Renderización inmediata del nuevo ítem en la pantalla principal de la app móvil del cliente.
2. Motor de Carrito y Compras	ID del producto seleccionado, cantidad deseada, modificadores (ej. "Sin cebolla").	Validación síncrona del stock en caché local. Multiplicación del precio unitario por cantidad, suma de tarifas de servicio (domicilio) y cálculo de subtotal.	Objeto JSON consolidado listo para ser enviado a la pasarela de pagos Wompi.
3. Procesamiento Transaccional	Token seguro de tarjeta de crédito o credenciales PSE, ID de la orden.	El sistema aísla los datos financieros enviándolos cifrados por HTTPS a Wompi. El backend local entra en modo de escucha asíncrona.	Webhook de confirmación. Cambio de estado en la BD de "Pendiente" a "Pagado" y generación de factura digital.

Módulo / Proceso	Datos de Entrada (Inputs)	Transformación y Lógica Interna	Salida Esperada (Outputs)
4. Enrutamiento Logístico	Coordenadas GPS (Latitud/ Longitud) generadas periódicamente por el hardware del móvil del repartidor.	El sistema inyecta las coordenadas en un canal WebSocket (Supabase Realtime) transmitiéndolas a los clientes suscritos a ese canal específico de la orden.	Actualización dinámica de la posición del marcador en el mapa de Google del cliente, recálculo del tiempo estimado (ETA).

4. Análisis de Límites del Sistema (system-analysis)

Para garantizar la seguridad y el desacoplamiento, se define una frontera estricta entre las responsabilidades del lado del cliente y el servidor:

- **Frontera del Cliente (Móvil y Web):** Responsable *únicamente* de la recolección de datos (inputs de usuario) y la renderización visual (UI). Toda regla matemática crítica (como el precio final a cobrar) se procesa en esta capa solo de manera visual, pero jamás se confía en ella para la transacción final.
- **Frontera del Servidor (Supabase & APIs externas):** Actúa como la fuente única de la verdad (Single Source of Truth). Recibe la petición de compra, recalcula los valores internamente consultando la tabla de precios para evitar manipulación (hacking) desde el móvil, y ejecuta la orden.
- **Frontera de Interacción Externa:** El sistema delega responsabilidades a cajas negras. El manejo del dinero recae 100% en el entorno de cumplimiento PCI-DSS de Wompi, liberando al proyecto de auditorías de seguridad bancaria.

5. Análisis Técnico y Justificación de Arquitectura (technical-analysis)

El análisis técnico profundo fundamenta por qué la arquitectura seleccionada es superior a los modelos tradicionales (como PHP/MySQL convencionales) para este caso de uso específico de delivery.

A. Eliminación del Polling mediante WebSockets (Realtime)

En plataformas antiguas, para saber si un pedido cambió de estado o dónde va el repartidor, la app debe hacer una petición a la base de datos cada 5 segundos (Polling), saturando el servidor. Nuestra arquitectura utiliza **Supabase Realtime**, que convierte los eventos de PostgreSQL en un flujo WebSocket constante. Si el restaurante marca "En Camino", la base de datos "empuja" (Push) la notificación directamente a la app del cliente en milisegundos, reduciendo el consumo de batería del móvil y ancho de banda en un 80%.

B. Prevención de Errores Críticos con TypeScript Strict Mode

El desarrollo se realizará bajo interfaces tipadas estrictas. Por ejemplo, al definir que el estado de un pedido solo puede ser el tipo unión `'PENDIENTE' | 'PREPARANDO' | 'EN_CAMINO' | 'ENTREGADO'`, el compilador de TypeScript impedirá que cualquier componente de la aplicación asigne un estado inválido. Esto garantiza que la app no colapse en tiempo de ejecución (Runtime Errors) cuando el cliente intente rastrear su pedido.

C. Seguridad de Endpoint mediante Firmas Criptográficas (Webhooks)

El proceso de confirmación de pago no dependerá de lo que reporte la app móvil, ya que esto podría ser falsificado por un usuario malintencionado. Se configurará un Endpoint en Next.js (Serverless Function) que recibirá notificaciones directas (Webhooks) desde los servidores de Wompi. Este endpoint utilizará un sistema de verificación de firma criptográfica mediante un *Secret Hash* para asegurar que la confirmación de pago proviene legítimamente del banco y no de un ataque externo antes de despachar la comida.

D. Estrategia de Gestión de Estado (State Management)

Para mantener el carrito de compras persistente incluso si el usuario cierra la aplicación, se utilizará un gestor de estado global asíncrono en React Native (como Zustand o Context API combinado con AsyncStorage). Esto mejora radicalmente la Experiencia de Usuario (UX), evitando frustraciones por pérdida de datos seleccionados.